“ALEXANDRU IOAN CUZA” UNIVERSITY OF IAŞI
FACULTY OF COMPUTER SCIENCE

BACHELOR’S THESIS

Sequre: A Real-Time Simulation and Visualization Platform for the BB84 Quantum Key Distribution Protocol

submitted by

Vezeteu Andrei

Session: July, 2026

Scientific coordinator
Conf. Dr. Andreea Arusoaie

Abstract

Quantum Key Distribution (QKD) promises information-theoretic security by encoding cryptographic key material into single photons whose state cannot be cloned or measured without disturbance. Despite this strong theoretical foundation, hands-on QKD experimentation is still difficult to access: existing simulators are either research-grade command-line tools written for specialists, or oversimplified notebooks that hide the very details (noise, finite-key effects, source imperfections, detector inefficiency, realistic attack strategies) that make QKD interesting and non-trivial.

This thesis presents **Sequre**, an interactive, full-stack platform for studying and visualising QKD. The system implements four protocols (BB84, B92, SARG04 and the entanglement-based E91) on top of a unified Qiskit [7] simulation backend, and also dispatches the same circuits to real superconducting hardware via the IBM Quantum runtime. The end-to-end pipeline is faithfully modelled: weak-coherent-pulse sources with three-intensity decoy-state analysis, photon-number-splitting and intercept-resend eavesdropping, depolarising and measurement noise, fibre attenuation at 0.20 dB/km, detector efficiency and dark counts, Cascade-style information reconciliation, Toeplitz-hash privacy amplification, finite-key correction in the Tomamichel–Lim–Gisin–Renner framework, and CHSH Bell tests for E91. Two machine-learning eavesdropper detectors — a Random Forest and a PyTorch LSTM trained on synthetic QBER sequences — demonstrate that data-driven channel monitoring is feasible even on small key blocks.

The front-end (React + Vite + Tailwind) streams qubit-level events from the FastAPI backend over WebSockets and presents them as a live photon grid, real-time QBER charts, decoy-state and finite-key panels, and a "Channel Status" badge. A gamified **Challenge Mode** adds fifteen procedurally-parameterised missions (Detective: detect Eve from observations; Engineer: configure QKD to satisfy a constraint) with a global XP leaderboard, persistent progress and SQLite-backed audit trails.

Experimental results confirm the expected QBER signatures of each attack type (intercept-resend $\approx 25\%$, weak Eve $\approx 12\%$, smart Eve tuned to 9%, PNS leaving QBER unchanged but collapsing the single-photon yield Y_1 in the decoy analysis), reproduce the $1/\sqrt{n}$ finite-key penalty, and yield validation accuracies of 0.93 (Random Forest) and 0.92 (LSTM) on held-out runs. Together, these results show that Sequre is both a pedagogically useful sandbox and a credible experimental platform for undergraduate-level QKD research.

Keywords: BB84, Quantum Key Distribution, QKD, Quantum Cryptography, Qiskit, IBM Quantum, FastAPI, React, WebSockets, QBER, Privacy Amplification, Error

Correction, No-Cloning Theorem.

Contents

1 Introduction

1.1 Motivation

The security of virtually all digital communication today rests on the computational difficulty of certain mathematical problems — factoring large integers in the case of RSA, and computing discrete logarithms in the case of elliptic-curve cryptography (ECC). These schemes are not proven secure in an information-theoretic sense; they are secure only because no classical polynomial-time algorithm is known to break them.

This foundation was shaken in 1994, when Peter Shor demonstrated that a sufficiently large quantum computer could factor an n-bit integer in polynomial time [2]. Although practical fault-tolerant quantum computers capable of breaking 2048-bit RSA remain several years away, the cryptographic community has already entered a transition period. The *harvest now, decrypt later* strategy — in which adversaries collect encrypted traffic today with the intent to decrypt it once quantum hardware matures — represents a concrete, near-term threat to the confidentiality of long-lived sensitive data [3].

In response to this threat, the National Institute of Standards and Technology (NIST) launched a post-quantum cryptography standardisation process in 2016, selecting its first set of quantum-resistant algorithms in 2024. However, *post-quantum cryptography* (PQC) still relies on mathematical hardness assumptions, merely choosing problems believed to be resistant to quantum algorithms. A fundamentally different approach, one that achieves *information-theoretic* security whose guarantees are grounded in the laws of physics rather than computational assumptions, is offered by **Quantum Key Distribution** (QKD).

QKD allows two parties, commonly referred to as Alice and Bob, to establish a shared secret key over an insecure channel such that any eavesdropping attempt by a third party (Eve) is detectable with overwhelming probability. The security follows from two foundational principles of quantum mechanics: the **no-cloning theorem**, which forbids copying an arbitrary unknown quantum state [4], and the disturbance introduced by measurement in an incompatible basis, as formalised by the **Heisenberg uncertainty principle**. Any interception by Eve necessarily disturbs the quantum states she measures, leaving a statistically detectable signature in the error rate of the received key.

The first and most influential QKD protocol was proposed by Charles Bennett and Gilles Brassard in 1984 [1]. The BB84 protocol encodes classical bits into the polarisation states of individual photons using two mutually unbiased bases, and its unconditional security has been rigorously proved under realistic noise models [5, 6]. Despite its theoretical

maturity, however, BB84 remains inaccessible to most students and researchers for practical experimentation: real QKD hardware is prohibitively expensive, and existing software simulators are either too narrowly focused on performance benchmarking, lack real-time visualisation, or do not integrate with actual quantum processors.

This accessibility gap motivates the present work. There is a clear need for an interactive, web-based platform that can demonstrate the full BB84 pipeline — from qubit generation through to a derived 128-bit secret key — while offering real hardware execution on IBM Quantum backends and providing a step-by-step visual intuition for every stage of the protocol. Such a tool would serve both an educational purpose, making quantum cryptography tangible for students encountering it for the first time, and a research purpose, providing a configurable testbed for studying protocol behaviour under varied noise regimes and adversarial conditions.

1.2 Proposed Solution

Sequre is a real-time simulation and visualisation platform for the BB84 Quantum Key Distribution protocol. It is implemented as a web application accessible through any modern browser, with no local installation required beyond a running Docker environment. The platform provides a complete, end-to-end demonstration of the quantum cryptographic pipeline under three distinct execution modes.

At its core, Sequre implements the full BB84 protocol in software using **Qiskit** [7] for quantum circuit construction and **Qiskit-Aer** for noise-model simulation. Alice encodes classical bits into single-qubit states drawn from the rectilinear basis ($|0\rangle$, $|1\rangle$) or the diagonal basis ($|+\rangle$, $|-\rangle$), and Bob measures each received qubit in a randomly chosen basis. Following the exchange, the sifting step retains only those bits where Alice and Bob chose the same basis (on average 50% of transmitted qubits). The platform then computes the **Quantum Bit Error Rate** (QBER), applies error correction via a parity-block reconciliation scheme, and performs privacy amplification by truncating a SHA-256 digest of the reconciled key to 128 bits.

The simulation operates in one of three modes selectable by the user:

- **Ideal simulation:** noise-free quantum circuits on a statevector or QASM simulator, producing a zero-QBER baseline.
- **Noisy simulation:** configurable depolarising and measurement errors applied via Qiskit-Aer, producing realistic QBER distributions that allow the user to study the protocol's behaviour as a function of hardware imperfection.
- **Real IBM Quantum hardware:** circuits are transpiled and dispatched to a real superconducting quantum processor via **Qiskit IBM Runtime** (SamplerV2 API),

currently supporting the `ibm_fez`, `ibm_marrakesh`, and `ibm_kingston` backends. Results are polled asynchronously and streamed live to the frontend.

Beyond the core protocol, Sequre implements several layers of physical and analytical realism that bring the simulator closer to a research-grade testbed:

- **Optical-fibre channel attenuation.** Each transmitted photon survives the channel with probability $\eta(L) = 10^{-\alpha L/10}$, where $\alpha = 0.2\,\mathrm{dB/km}$ is the standard single-mode-fibre loss coefficient and L is the configurable transmission distance. A dedicated chart plots the asymptotic secure key rate $R \sim \eta(L)(1 - 2h(Q))$ across 0–200 km, illustrating the well-known rate–distance limits of point-to-point QKD without trusted nodes.
- **Decoy-state BB84 with weak coherent pulses.** The user can switch from an idealised single-photon source to a *weak-coherent-pulse* (WCP) source emitting two intensity classes (signal μ_s and decoy μ_d), with Poissonian photon-number statistics. Per-intensity gains and error rates are tracked online, and the GLLP two-decoy bounds are evaluated to estimate the single-photon yield Y_1 and error rate e_1 — the quantities required for security against the photon-number-splitting (PNS) attack.
- **Full CASCADE error correction.** The original parity-block scheme was replaced with the multi-pass CASCADE algorithm with back-correction, exposing both the corrected key and the information leakage ℓ that is later consumed by privacy amplification.
- **Machine-learning eavesdropper detection.** A Random Forest classifier trained on six contextual features per run (QBER, sifting efficiency, depolarising noise, measurement-error rate, channel distance, protocol length) produces an estimated probability that an eavesdropper was present. The classifier is retrainable on demand from the SQLite simulation history and provides a security signal that operates orthogonally to the QBER threshold.
- **User accounts and per-user attribution of runs.** A full authentication layer (email/password with bcrypt, JWT-based sessions in HttpOnly cookies, CSRF double-submit protection, password reset, and Google OAuth 2.0 sign-in) allows simulation runs to be attributed to authenticated users while keeping anonymous access fully functional.

An optional **Eve eavesdropping simulation** implements three distinct attack strategies of progressively increasing sophistication: *Weak Eve*, which intercepts 30% of transmitted qubits; *Strong Eve*, which intercepts all qubits and reliably generates a QBER of approximately 25%, well above the 11% security threshold; and *Smart Eve*, an adaptive intercept-resend adversary that uses a proportional feedback loop on the

running QBER to tune its interception probability towards a user-configurable target (e.g. 9%, just below the abort threshold), thereby evading a naive QBER-only security check. The Smart Eve mode motivates the machine-learning detector described above and provides a concrete adversarial baseline against which the classifier is evaluated.

The user interface, built with **React** (Vite) and **Tailwind CSS**, receives per-qubit events from the backend over a persistent WebSocket connection and renders them live without page refresh. The **PhotonGrid** component animates the first 80 qubit exchanges cell by cell, encoding each cell's basis symbol, bit value, and Bob's measurement basis, and highlights Eve-intercepted qubits with a visual marker. A **key distillation pipeline** component visualises the progressive shrinkage of the key through each post-processing stage, while the **QBER historical chart** plots error rates across all previous simulation runs against the security threshold line. An **educational panel** of five expandable cards introduces the underlying theory in accessible language alongside the simulation results.

Completed simulation runs are automatically persisted to a **SQLite** database, and REST endpoints allow history retrieval and selective deletion, enabling longitudinal analysis of QBER behaviour across multiple experiments.

1.3 Contributions and Impact

The primary contributions of this work are the following:

C1. A fully functional, end-to-end BB84 implementation on real quantum hardware. Unlike purely classical simulators, Sequre executes single-qubit BB84 circuits on IBM superconducting processors via the Qiskit IBM Runtime SamplerV2 interface. This integration enables the direct observation of real decoherence effects — gate errors, readout noise, crosstalk — on the QKD error rate, a dimension that no software-only simulator can replicate.

C2. Real-time, event-driven visualisation of the quantum protocol. The WebSocket streaming architecture delivers per-qubit simulation events to the frontend as they are generated, producing a live, animated view of the key exchange. This level of temporal granularity is absent from existing QKD educational tools, which typically present only aggregate results after a simulation completes.

C3. A spectrum of eavesdropping models, from passive to adaptive. Beyond the textbook intercept-resend Eve, Sequre implements a *Smart Eve* adversary that uses a proportional feedback loop on the running QBER to evade the standard abort threshold while still extracting partial information. Combined with the QBER-based security check, this stack of three Eve modes (Weak, Strong, Smart)

demonstrates concretely both the security guarantees and the *limitations* of a threshold-only check against an intelligent adversary.

C4. A realistic optical-fibre channel model with rate–distance curves. A configurable attenuation coefficient ($\alpha = 0.2\,\mathrm{dB/km}$ by default) is applied per qubit, and the asymptotic secure key rate is plotted as a function of channel length, bringing the simulator in line with standard QKD benchmarking methodology.

C5. Decoy-state BB84 with weak-coherent-pulse modelling. Two intensity classes (signal and decoy) are emitted with Poissonian photon-number statistics, and the GLLP two-decoy bounds are evaluated online to recover the single-photon yield and error rate — the security-critical quantities under the PNS attack.

C6. Full CASCADE error correction with information-leakage accounting. The multi-pass binary-search reconciliation with back-correction replaces the lossy parity-block scheme and exposes the leakage ℓ that feeds into privacy amplification, closing the loop on a rigorous post-processing pipeline.

C7. Supervised eavesdropper detection via machine learning. A Random Forest classifier trained on six contextual features per run produces a probability of eavesdropper presence that is complementary to (and orthogonal in failure mode to) the QBER threshold, providing a defence against precisely the adaptive adversaries that the QBER check cannot catch on its own.

C8. Multi-user platform with authenticated session management. A full authentication layer (email/password with bcrypt, JWT access/refresh tokens in HttpOnly cookies, CSRF double-submit protection, password reset, Google OAuth 2.0) allows simulations to be attributed to authenticated users and enables per-user progress tracking across the gamified Challenge Mode, while preserving full anonymous guest access for the simulation features.

C9. A configurable research testbed for QKD experimentation. The platform exposes noise parameters (depolarising probability, measurement error rate, interception fraction, channel distance, source type, decoy intensities, error-correction algorithm, detector efficiency η_{det}, dark-count rate) as user-controlled inputs, together with a reproducibility seed for deterministic runs and seven preset scenarios that instantly configure canonical experimental setups. The platform is suitable not only for classroom demonstration but also as an exploratory tool for studying protocol behaviour under varied hardware and adversarial conditions.

C10. Persistent simulation history with REST API access. Automated SQLite

persistence of every simulation run, queryable via REST endpoints, enables longitudinal analysis and reproducible comparison of results across hardware modes, noise levels, and Eve configurations, while also serving as the training corpus for the ML classifier.

C11. Multi-protocol support: B92, SARG04, and E91 with CHSH Bell test. Three additional QKD protocols are implemented alongside BB84. B92 uses only two non-orthogonal states and conclusive-outcome sifting ($\approx 25\%$ efficiency). SARG04 matches BB84's four states but announces *pairs* of states at sifting time, providing resistance against the PNS attack without requiring a decoy source. E91 generates entangled Bell pairs ($|\Phi^+\rangle$), distributes one photon to each party, and derives the key from the correlated measurement outcomes; eavesdropping is certified by computing the CHSH correlator S and verifying $|S| > 2$, the classical bound. The platform visualises S in real time and flags Bell-inequality violation or collapse.

C12. Explicit Photon-Number-Splitting attack simulator. A dedicated Eve mode models the PNS attack on WCP sources: single-photon pulses are blocked (no information obtainable without disturbing the state), while multi-photon pulses are split with one photon retained in quantum memory and the rest forwarded losslessly. The resulting collapse of the single-photon yield $Y_1 \to 0$ and the secure key rate $R \to 0$, as computed by the Lo-Ma-Chen decoy-state bounds, is displayed in the PNS Attack panel alongside the fraction of pulses Eve successfully intercepted.

C13. Composable finite-key security bounds (Tomamichel 2012). A dedicated analysis module computes the Hoeffding statistical correction $\delta_{\mathrm{PE}} = \sqrt{\ln(2/\varepsilon_{\mathrm{PE}})/(2n)}$ on the QBER estimate, the Slepian–Wolf error-correction leakage, and the privacy-amplification overhead, yielding both the asymptotic (textbook GLLP) key length and the rigorous composable finite-key length for the actual block size n. The panel visualises the gap between the two and identifies the minimum n required for a positive finite-key bound at a given security parameter $\varepsilon_{\mathrm{sec}}$.

C14. LSTM deep-learning eavesdropper detector. A recurrent neural network (PyTorch, hidden size 64, one LSTM layer) is trained on 1 200 synthetic per-qubit exchange sequences and learns to predict $P(\text{Eve} \mid \text{sequence})$ from the full temporal pattern of alice/bob basis choices, bob results, and error flags. This model achieves 92% validation accuracy and correctly flags intercept-resend attacks at the operational 30% interception rate — a regime in which the QBER remains below the 11% abort threshold and a threshold-only check would declare the channel secure. Model weights are cached on disk and loaded at startup; training

completes in under 30 seconds on CPU.

C15. Gamified single-player Challenge Mode with global leaderboard. A self-contained *Challenge Mode* page presents 15 procedurally parameterised QKD missions across two pedagogical formats. *Detective* missions run a hidden simulation and ask the student to declare whether Eve is present and which attack she is using, based solely on the observable panels (QBER, LSTM probability, decoy-state Y_1, CHSH S). *Engineer* missions present a constrained objective (e.g. "achieve ≥ 256 bits of finite-key output over a 130 km link under PNS") and ask the student to configure the QKD parameters to meet it. Each mission is parameterised within a difficulty band on every attempt; a global XP leaderboard, per-user progress tracking, and targeted post-failure hints complete the learning loop. The Challenge Mode reuses the entire simulation and visualisation stack without duplication, and is implemented as a separate `/challenge` route with its own authenticated REST API.

Beyond its immediate academic scope, Sequre is conceived as a research-oriented testbed with a long-term ambition: to probe the practical limits of quantum key distribution under realistic hardware imperfections, advanced attack models, and extended protocol variants. The architecture is deliberately modular — the quantum logic, classical post-processing, and presentation layers are cleanly separated — and this modularity has enabled the integration of decoy-state BB84, CASCADE error correction, an optical-fibre attenuation model, an adaptive (Smart) Eve, an ML-based detection layer, three alternative QKD protocols (B92, SARG04, E91 with CHSH Bell test), an explicit PNS-attack simulator, composable finite-key security bounds, a deep-learning (LSTM) eavesdropper detector, and a fully gamified *Challenge Mode* into the same code base without restructuring.

Additional physical-realism controls exposed at the user level include configurable **detector efficiency** η_{det} and **dark-count rate**, both of which modulate Bob's measurement outcome in accordance with the standard detector imperfection model; an optional **reproducibility seed** that seeds the NumPy generator for bit-for-bit reproducible runs; and a set of **preset scenarios** (Lab bench, Metropolitan link, Satellite uplink under PNS, Adversarial) that configure all simulation parameters atomically for use in demonstrations and thesis presentations. The WCP + decoy-state source and PNS attack model have been extended to cover the B92 and SARG04 alternative protocols, enabling direct side-by-side comparison with BB84.

2 Theoretical Background

This chapter provides the theoretical foundations necessary to understand the Sequre platform and the BB84 protocol it implements. It begins with a comparison of classical and quantum approaches to cryptographic key distribution, introduces the relevant quantum mechanical formalism, and then describes in detail the BB84 protocol, its security metric (QBER), and the classical post-processing steps that transform raw qubit measurements into a usable secret key.

2.1 Classical vs. Quantum Cryptography

Classical cryptography can be divided into two broad categories: *symmetric-key* cryptography, in which sender and receiver share a common secret key, and *public-key* (asymmetric) cryptography, in which a public key is used for encryption and a private key for decryption. The security of public-key schemes such as RSA and elliptic-curve cryptography (ECC) relies on the computational intractability of certain mathematical problems — integer factorisation and the discrete logarithm problem, respectively. These problems are hard for classical computers, but as established in Section 1.1, Shor's algorithm [2] solves both in polynomial time on a sufficiently powerful quantum computer. Symmetric schemes are weakened but not broken: Grover's algorithm reduces an n-bit key search from $O(2^n)$ to $O(2^{n/2})$ [8], so doubling the key length restores the original security margin.

Quantum Key Distribution takes an entirely different approach. Rather than relying on mathematical assumptions, QKD uses the physical properties of quantum states to distribute a key in such a way that any interception attempt is detectable in principle. The security is *information-theoretic*: it holds regardless of the computational power of the adversary. The key insight is that quantum mechanics imposes a fundamental constraint on any measurement process — measuring a quantum state in an incompatible basis irreversibly disturbs that state, leaving a detectable trace.

Table 1 summarises the key differences between the two paradigms.

Table 1 – Classical vs. quantum key distribution — a comparison.

Property	**Classical (RSA/ECC)**	**QKD (BB84)**
Security basis	Computational hardness	Laws of quantum physics
Quantum threat	Broken by Shor's algorithm	Unaffected
Eavesdrop detection	Not inherent	Guaranteed by physics
Key distribution	Public-key exchange	Quantum channel + public classical channel
Security model	Computational	Information-theoretic

2.2 Quantum Mechanics Primer

2.2.1 Qubits and Quantum States

The fundamental unit of quantum information is the **qubit** (quantum bit). Unlike a classical bit, which is either 0 or 1, a qubit can exist in a *superposition* of both basis states simultaneously. Using Dirac (bra-ket) notation, the general state of a qubit is written as:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1, \tag{1}$$

where $|0\rangle$ and $|1\rangle$ are the computational basis states (eigenstates of the Pauli-Z operator), and $|\alpha|^2$, $|\beta|^2$ are the probabilities of measuring the qubit in state $|0\rangle$ or $|1\rangle$, respectively.

2.2.2 Measurement and State Collapse

A key property of quantum mechanics is that measuring a qubit in a given basis *collapses* its state to one of the basis vectors. If the qubit is in state $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ and we measure in the Z-basis $\{|0\rangle\,, |1\rangle\}$, we obtain outcome 0 with probability $|\alpha|^2$ and outcome 1 with probability $|\beta|^2$, and the state collapses accordingly. After the measurement, all information about the pre-measurement superposition is irrevocably lost.

Crucially, if a qubit is prepared in a state belonging to one orthonormal basis and then measured in a *different*, incompatible basis, the outcome is uniformly random and the original state cannot be inferred. This is the physical mechanism exploited by BB84 to detect eavesdropping.

2.2.3 The Hadamard Gate and the Diagonal Basis

In addition to the computational Z-basis $\{|0\rangle, |1\rangle\}$, BB84 uses the *diagonal* (X-basis) $\{|+\rangle, |-\rangle\}$, defined by:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \tag{2}$$

The transformation between the two bases is performed by the **Hadamard gate** H:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle. \tag{3}$$

In the Sequre implementation, the Hadamard gate is applied in Qiskit via `qc.h(0)` when Alice encodes a bit in the X-basis, and equally when Bob measures in the X-basis (rotating back before the standard Z-measurement).

2.3 The No-Cloning Theorem

> **Theorem** (Wootters and Zurek, 1982 [4]): There is no unitary operation U that can copy an arbitrary unknown quantum state. Formally, there exists no U such that $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$ for all $|\psi\rangle$.

Proof sketch. Suppose such a U existed. Then for any two states $|\psi\rangle$ and $|\phi\rangle$:

$$U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle, \tag{4}$$

$$U(|\phi\rangle|0\rangle) = |\phi\rangle|\phi\rangle. \tag{5}$$

Taking the inner product of both sides:

$$\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2. \tag{6}$$

This equation holds only if $\langle\psi|\phi\rangle = 0$ (orthogonal states) or $\langle\psi|\phi\rangle = 1$ (identical states). For any other pair of states, $0 < |\langle\psi|\phi\rangle| < 1$, so no such U can exist. □

Implication for QKD. The no-cloning theorem makes it impossible for Eve to intercept a transmitted qubit, make a perfect copy, forward the original to Bob, and measure the copy at her leisure. Any attempt by Eve to gain information about the qubit requires her to interact with it, which — as the following section explains — necessarily disturbs the quantum state.

2.4 Heisenberg's Uncertainty Principle in QKD

Heisenberg's uncertainty principle states that certain pairs of physical properties, known as *complementary observables*, cannot both be known simultaneously with arbitrary precision. In the context of BB84, the relevant complementary observables are the Pauli-Z and Pauli-X operators, corresponding to measurement in the computational basis and the diagonal basis, respectively.

Consider a qubit sent by Alice in the Z-basis in state $|0\rangle$. If Bob — or Eve — measures this qubit in the X-basis instead, the outcome is uniformly random: $|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$, so the result is $+$ or $-$ with equal probability $\frac{1}{2}$. Moreover, after this X-basis measurement, the qubit's state has collapsed to either $|+\rangle$ or $|-\rangle$ — no longer $|0\rangle$. If this disturbed qubit is forwarded to Bob and he measures in the Z-basis (as Alice intended), his result is again uniformly random, with a 50% probability of disagreeing with Alice's original bit.

This is the precise mechanism by which eavesdropping is detected in BB84:

- When Eve intercepts a qubit, she must choose a measurement basis without knowing which basis Alice used.
- She guesses the correct basis with probability $\frac{1}{2}$.
- When she guesses wrong (50% of intercepts), her measurement disturbs the state.
- Of those disturbed qubits, Bob's measurement disagrees with Alice's original bit with probability $\frac{1}{2}$.

Each intercepted qubit therefore introduces an error with probability $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$. A complete intercept-resend attack (Strong Eve) thus raises the QBER to approximately 25%.

2.5 The BB84 Protocol

The BB84 protocol [1] proceeds as follows.

Step 1 — Qubit preparation (Alice). Alice generates a random bit string $\mathbf{a} = (a_1, \ldots, a_n)$ and an independent random basis string $\mathbf{x} = (x_1, \ldots, x_n)$, where each $x_i \in \{Z, X\}$. She encodes each bit a_i into a qubit according to Table 2 and transmits the qubits one by one over the quantum channel.

Table 2 – BB84 encoding: the four qubit states used in the protocol.

Bit	Basis	State	Description
0	Z (rectilinear)	$\|0\rangle$	spin-up / horizontal polarisation
1	Z (rectilinear)	$\|1\rangle$	spin-down / vertical polarisation
0	X (diagonal)	$\|+\rangle = \frac{1}{\sqrt{2}}(\|0\rangle + \|1\rangle)$	+45ř polarisation
1	X (diagonal)	$\|-\rangle = \frac{1}{\sqrt{2}}(\|0\rangle - \|1\rangle)$	−45ř polarisation

Step 2 — Qubit measurement (Bob). Bob independently chooses a random basis string $\mathbf{y} = (y_1, \ldots, y_n)$ and measures each received qubit in the corresponding basis, recording his outcomes $\mathbf{b} = (b_1, \ldots, b_n)$.

Step 3 — Basis sifting. Alice and Bob communicate their basis choices $\mathbf{x}$ and $\mathbf{y}$ over a public classical channel (no bit values are revealed). They retain only the positions where $x_i = y_i$ — on average, 50% of all transmitted qubits. The retained bits form the *sifted key*. In Sequre, this step is implemented in `classical_processing/sifting.py`:

```
def sift_keys(alice_bits, alice_bases, bob_bits, bob_bases):
    alice_key, bob_key = [], []
    for a_bit, a_basis, b_bit, b_basis in zip(
            alice_bits, alice_bases, bob_bits, bob_bases):
        if a_basis == b_basis:
            alice_key.append(a_bit)
            bob_key.append(b_bit)
    return alice_key, bob_key
```

Listing 1: Basis sifting in Sequre.

Step 4 — Error rate estimation (QBER). Alice and Bob sacrifice a random subset of their sifted key to estimate the Quantum Bit Error Rate (QBER). If QBER exceeds a security threshold, they abort the protocol (see Section 2.6).

Step 5 — Error correction. Alice and Bob apply an error-correction protocol over the public channel to reconcile the remaining bits of their sifted keys (see Section 2.7).

Step 6 — Privacy amplification. To account for any partial information Eve may have obtained, Alice and Bob apply a universal hash function to compress their reconciled key into a shorter but statistically independent final key (see Section 2.7).

2.6 Quantum Bit Error Rate (QBER)

The **Quantum Bit Error Rate** (QBER) is defined as the fraction of sifted key bits on which Alice and Bob disagree:

$$Q = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[a_i \neq b_i], \tag{7}$$

where N is the length of the sifted key and a_i, b_i are the corresponding bits held by Alice and Bob after sifting.

In a noise-free channel without eavesdropping, $Q = 0$. Channel noise (depolarising errors, measurement errors) raises Q by a small, bounded amount. An active intercept-resend attack raises Q significantly above any noise floor. Specifically:

- **No Eve, no noise:** $Q \approx 0$.
- **Weak Eve** (intercepts fraction p of qubits): each intercepted qubit introduces an error with probability $\frac{1}{4}$ (wrong basis with probability $\frac{1}{2}$, error given wrong basis with probability $\frac{1}{2}$), so $Q \approx \frac{p}{4}$. For Sequre's Weak Eve ($p = 0.30$), this gives $Q \approx 7.5\%$.
- **Strong Eve** ($p = 1.0$, full intercept-resend): $Q \approx 25\%$.

Sequre uses a security threshold of $Q_{\text{th}} = 11\%$, consistent with the standard BB84 security analysis [9]: if $Q \geq 11\%$, the protocol is aborted and no key is output. This threshold ensures that any remaining information Eve holds about the key can be made exponentially small by privacy amplification.

2.7 Error Correction and Privacy Amplification

2.7.1 Error Correction

Even in the absence of eavesdropping, channel noise causes a non-zero QBER. Before a key can be used, Alice and Bob must reconcile their sifted keys so that they hold identical bit strings. This is performed via an *information reconciliation* protocol over the public classical channel.

Sequre implements a simplified parity-block scheme: the sifted key is divided into consecutive blocks of $B = 4$ bits. For each block, Alice announces the block's parity (XOR of its bits) over the public channel. Bob computes the parity of his corresponding block. If the parities agree, both parties retain the block. If they disagree (indicating at least one error within the block), the entire block is discarded by both parties. The implementation in `classical_processing/error_correction.py` is:

```
1 def correct_errors(alice_key, bob_key, block_size=4):
```

```
    corrected_alice, corrected_bob = [], []
    for i in range(0, len(alice_key) - block_size + 1, block_size):
        a_block = alice_key[i:i + block_size]
        b_block = bob_key[i:i + block_size]
        if parity(a_block) == parity(b_block):
            corrected_alice.extend(a_block)
            corrected_bob.extend(b_block)
    return corrected_alice, corrected_bob
```

Listing 2: Parity-block error correction in Sequre.

This approach sacrifices key material (entire blocks are discarded rather than individual erroneous bits), but guarantees that all retained blocks are error-free with high probability for low to moderate QBER. The full **CASCADE** algorithm [11] improves upon this by iteratively narrowing down the position of individual erroneous bits through binary search, achieving near-optimal reconciliation efficiency. CASCADE is identified as a future development direction for Sequre (see Section 8).

2.7.2 Privacy Amplification

After error correction, Alice and Bob share an identical key, but Eve may hold partial information about it: she observed parity announcements over the public channel during error correction, and she may have obtained information from any qubits she intercepted before the QBER check. Privacy amplification *compresses* the reconciled key using a hash function, reducing Eve's information about the output key to a negligible quantity regardless of how much she learned about the input [10].

Sequre implements privacy amplification by computing the **SHA-256** hash of the reconciled key and truncating the digest to 128 bits (32 hexadecimal characters):

```
def amplify_privacy(key, output_bits=128):
    byte_length = (len(key) + 7) // 8
    key_int = int("".join(str(b) for b in key), 2) if key else 0
    key_bytes = key_int.to_bytes(byte_length, byteorder="big")
    digest = hashlib.sha256(key_bytes).hexdigest()
    return digest[: output_bits // 4]  # 32 hex chars = 128 bits
```

Listing 3: SHA-256 privacy amplification in Sequre.

SHA-256 is a collision-resistant, one-way function and serves as a practical approximation to the universal hash families used in the information-theoretic proofs of privacy amplification. While not strictly optimal from a universal-hashing standpoint — optimal constructions use seeded hash families whose seed is shared publicly — SHA-256 truncation is a common and accepted choice for prototype implementations, and provides 128 bits of output security in the absence of attacks on the hash function itself.

2.8 Optical-Fibre Channel Attenuation

The discussion so far has treated the quantum channel as a passive conduit that may add gate-level noise but transmits every qubit. In any physical implementation of QKD, however, the dominant cause of bit loss is not noise but *photon attenuation* along the channel. For optical fibre links, the survival probability of a photon transmitted over a distance L (in km) follows the Beer–Lambert law:

$$\eta(L) = 10^{-\alpha L/10}, \tag{8}$$

where α is the fibre's loss coefficient. The de-facto industry value for telecom-grade single-mode fibre at the 1550 nm wavelength is $\alpha \approx 0.2\,\mathrm{dB/km}$ [12]. This implies, for instance, that only $\eta(50) \approx 10\%$ of photons survive a 50-km link and $\eta(100) \approx 1\%$ survive 100 km. Free-space links incur instead *beam-divergence* loss, modelled as $\eta \propto 1/L^2$ but with substantially smaller coefficients per unit distance over short ranges.

The practical consequence for QKD is that the *secure key rate* — the number of secret-key bits produced per channel use — decays exponentially with distance. In the asymptotic (large-N) regime, the secret-key rate for BB84 with one-way classical communication is given by the GLLP–Shor–Preskill formula [9, 13]:

$$R(L) \;\geq\; \eta(L)\Big[1 \;-\; H_2(e_b) \;-\; f(e_b)\,H_2(e_b)\Big], \tag{9}$$

where e_b is the bit-error rate, H_2 is the binary entropy $H_2(x) = -x\log_2 x - (1-x)\log_2(1-x)$, and $f(e_b) \geq 1$ is the efficiency factor of the error-correction algorithm relative to the Shannon limit. The term $1 - H_2(e_b)$ accounts for the information Eve could have gained from the quantum channel; the term $f(e_b)H_2(e_b)$ discounts the bits revealed during classical reconciliation.

Equation (9) sets the well-known *rate–distance trade-off* of BB84: there exists a maximum distance L^* beyond which the right-hand side becomes negative and no positive key rate can be obtained. For $\alpha = 0.2\,\mathrm{dB/km}$, $e_b = 1\%$, and CASCADE-typical $f \approx 1.16$, $L^* \approx 150$–$200\,\mathrm{km}$. Surpassing this limit requires either trusted intermediate nodes, twin-field protocols [14], or quantum repeaters — a key observation that motivates the Sequre rate-vs-distance visualisation in Section 6.

2.9 Decoy-State Method and the Photon-Number-Splitting Attack

The BB84 protocol as originally formulated assumes Alice transmits a *single-photon source* (SPS), so that any attempt by Eve to extract information necessarily perturbs

the qubit. In practice, true single-photon sources are demanding to engineer; instead, the overwhelming majority of deployed QKD systems use attenuated lasers — so-called *weak coherent pulses* (WCP). A WCP with mean photon number μ emits, per pulse, n photons with probability:

$$P(n \mid \mu) \;=\; e^{-\mu}\,\frac{\mu^{n}}{n!}. \tag{10}$$

For $\mu < 1$, the dominant contributions are $n = 0$ (vacuum) and $n = 1$ (single photon), but a non-negligible fraction of pulses contains $n \geq 2$ photons. These multi-photon pulses expose the protocol to a **photon-number-splitting** (PNS) attack: Eve installs a quantum non-demolition photon-number measurement at the output of Alice, blocks all $n = 1$ pulses, and for every $n \geq 2$ pulse keeps one photon in a quantum memory while forwarding the rest to Bob unaltered. After Alice and Bob publicly announce their bases, Eve measures her stored photons in the correct basis, obtaining the bit value with no QBER penalty whatsoever.

The PNS attack is fatal to naive WCP-BB84 over high-loss channels, because in the limit $\eta(L) \to 0$ the legitimate single-photon signal can be completely suppressed in favour of intercepted multi-photon contributions without raising the QBER above the security threshold. The **decoy-state method** [15, 16] elegantly defeats this attack by having Alice randomly switch among several intensity classes:

- a *signal* intensity μ_s used for key generation;
- one or more *decoy* intensities $\mu_d, \nu, \ldots$ used only for monitoring.

Because Eve does not know in advance the intensity of any individual pulse, she cannot distinguish signal multi-photon pulses from decoy multi-photon pulses. The per-intensity gains $Q_\mu = \sum_n P(n \mid \mu) Y_n$ and error rates E_μ provide a system of linear inequalities that bound the *single-photon yield* Y_1 and *single-photon error rate* e_1 in a verifiable way. In the two-decoy GLLP analysis, the lower bound on Y_1 is:

$$Y_1^{L} \;\geq\; \frac{\mu}{\mu\nu - \nu^{2}}\left(Q_\nu e^{\nu} - Q_\mu e^{\mu}\,\frac{\nu^{2}}{\mu^{2}} - \frac{\mu^{2} - \nu^{2}}{\mu^{2}}\,Y_0\right), \tag{11}$$

where μ and ν are the signal and decoy means and Y_0 is the background (dark-count) rate. The secure key rate is then:

$$R \;\geq\; q\left[-Q_\mu f(E_\mu) H_2(E_\mu) + Q_1^{L}\left(1 - H_2(e_1^{U})\right)\right], \tag{12}$$

with $Q_1^{L} = \mu e^{-\mu} Y_1^{L}$. The Sequre decoy-state panel renders each of these quantities live, so the student can watch the single-photon contribution to the key being recovered from the otherwise aggregate WCP statistics.

2.10 The CASCADE Information-Reconciliation Algorithm

The block-parity reconciliation described in Section 2.7 is a useful pedagogical baseline but is highly suboptimal: it discards entire blocks on any disagreement, leaking only $\log_2(B)$ bits less than the optimum per corrected error but losing B bits of key in the process. The CASCADE algorithm [11], introduced by Brassard and Salvail in 1993, remains the de-facto reference for QKD error correction to this day because it isolates and corrects individual erroneous bits rather than discarding their surrounding blocks.

CASCADE proceeds in k *passes*, each pass parameterised by a block size B_i. The first block size is chosen as a function of the estimated QBER Q:

$$B_1 \approx \left\lceil \frac{0.73}{Q} \right\rceil, \qquad B_{i+1} = 2\,B_i. \tag{13}$$

Within a pass, the key is permuted by a fresh random permutation π_i and partitioned into consecutive blocks of size B_i. Alice announces the parity of each block over the public channel; for every block whose parity differs from Bob's, the parties perform a binary search — recursively bisecting the block and comparing parities of the halves — until a single erroneous bit is located and flipped.

The crucial element of CASCADE is its **back-correction** mechanism: flipping a bit in pass i changes the parities of every block in passes $1, \ldots, i-1$ that contained that bit. A flipped bit in a previously "correct" block must have masked a second error. CASCADE therefore revisits all earlier blocks whose parity has now changed and continues binary-searching for the remaining errors. This cross-pass propagation allows the algorithm to converge with high probability after only four to six passes, leaking

$$\ell \approx N \cdot f(Q) \cdot H_2(Q) \tag{14}$$

bits over the public channel, with $f(Q) \approx 1.05$–1.22 for typical QKD QBER values — close to the Shannon lower bound of $f = 1$ [17]. The leakage ℓ is precisely the quantity subtracted during privacy amplification to obtain the final secret-key length.

In Sequre, CASCADE is implemented in `backend/classical_processing/cascade.py` and replaces the block-parity scheme. The legacy scheme remains available behind a UI selector for didactic comparison: a side-by-side view shows how many bits are wasted by block discarding versus how many leak through CASCADE parities at the same input QBER.

2.11 Machine Learning for QKD Security Monitoring

A QBER-threshold check is a powerful but coarse instrument. It catches any adversary that exceeds the abort threshold but is, by construction, blind to attacks calibrated to stay below it — attacks that Sequre's Smart Eve mode produces by design. A complementary, statistics-driven detector is therefore desirable.

The detection of subtle adversarial behaviour from an experiment's contextual signature (QBER, sifting efficiency, noise parameters, distance, sample size) is a textbook *binary classification* problem: given a feature vector $\mathbf{x} \in \mathbb{R}^d$, predict the label $y \in \{0, 1\}$ (*Eve absent*, *Eve present*). Among classical classifiers, the **Random Forest** [18] is particularly well-suited to this setting: it is a non-linear ensemble of decision trees that handles feature interactions natively, is robust to the heterogeneous, non-Gaussian distributions characteristic of QKD features, exposes interpretable feature importances, and trains in seconds on the dataset sizes (10^2–10^3 runs) typical of an educational platform.

Concretely, the model computes

$$\hat{P}(y = 1 \mid \mathbf{x}) \;=\; \frac{1}{T} \sum_{t=1}^{T} \hat{P}_t(y = 1 \mid \mathbf{x}), \tag{15}$$

where each $\hat{P}_t$ is the leaf-frequency estimate of the t-th tree trained on a bootstrap sample of the historical runs. The thesis uses $T = 100$ trees with the scikit-learn default depth limits. Labels are derived automatically from the configured `eve_mode` of each historical run: anything other than `eve_mode == 'none'` is treated as a positive instance.

While the Random Forest operates on six aggregate features per run, the literature has demonstrated that deeper models can exploit the full temporal structure of the qubit exchange. LSTM networks, which process the per-qubit sequence as a time series, have been shown to reach $> 90\%$ accuracy against adaptive attackers in CV-QKD [19]. Sequre implements both approaches: the Random Forest for fast, interpretable per-run scoring and the LSTM detector described below for sequence-level detection of subtle patterns that summary statistics flatten away.

2.11.1 LSTM Per-Qubit Sequence Detector

The Random Forest classifier described above is trained once on the accumulated simulation history and operates on six hand-engineered aggregate features. While effective against obvious eavesdropping, it cannot exploit the *temporal ordering* of qubit-level events — a dimension that is informative under intercept-resend attacks,

where errors cluster on basis-matched positions where Eve chose the wrong basis, rather than appearing uniformly at random.

To address this, Sequre includes a **Long Short-Term Memory (LSTM)** classifier [24] that reads the entire per-qubit exchange as an ordered sequence. Each qubit i produces a six-dimensional feature vector:

$$\mathbf{x}_i = \Big(a_i^{\text{basis}},\ b_i^{\text{basis}},\ a_i^{\text{bit}},\ b_i^{\text{result}},\ \mathbf{1}[a_i^{\text{basis}} = b_i^{\text{basis}}],\ \mathbf{1}[b_i^{\text{result}} \neq a_i^{\text{bit}}] \cdot \mathbf{1}[a_i^{\text{basis}} = b_i^{\text{basis}}]\Big), \quad (16)$$

where the last two components encode the basis-match indicator and the error indicator restricted to basis-matched positions, respectively. These are the positions Eve's intercept-resend attack contaminates.

The network architecture is compact by design — CPU inference is required during the live simulation stream:

- **Input:** variable-length sequence $[\mathbf{x}_1, \ldots, \mathbf{x}_n] \in \mathbb{R}^{n \times 6}$
- **LSTM layer:** hidden size 64, one layer, batch-first
- **Pooling:** element-wise sum of the mean-pooled hidden states and the final hidden state h_n, yielding a 64-dimensional representation
- **Fully connected:** $64 \to 2$ (logits for *secure* / *compromised*)

Synthetic training data. Because real QKD runs accumulate slowly during a session, the model is trained offline on a *synthesised* corpus of 1 200 sequences. Each sequence is generated by rolling the per-qubit BB84 exchange stochastically: for Eve-present samples, the interception rate is drawn from $[0.25, 1.0]$ with a bias towards the boundary regime $[0.25, 0.45]$ — the hardest cases for a threshold-only check. For Eve-absent samples, the channel noise is drawn uniformly from $[0.5\%, 5\%]$. An additional 200 sequences are held out for validation.

Training and results. The model is trained for 60 epochs with the Adam optimiser ($\eta = 2 \times 10^{-3}$) and cross-entropy loss. On the held-out validation set the model achieves:

- **Validation accuracy:** 92%
- **Validation loss:** 0.181
- **Training accuracy:** 89.5%

Critically, at the operational Weak Eve rate of 30% interception (which produces QBER $\approx 7.5\%$, below the 11% abort threshold), the LSTM reports $P(\text{Eve}) > 0.97$ consistently across repeated trials, whereas the QBER check alone would declare the channel secure.

Weights are persisted to `backend/ml/lstm_weights.pt` and loaded at server startup; if

no cached weights exist, training runs automatically in the lifespan hook and completes in under 30 seconds on CPU.

2.12 Alternative QKD Protocols

BB84 is the reference QKD protocol, but several variants have been proposed to address different hardware constraints and threat models. Sequre implements three alternatives.

2.12.1 B92 (Bennett, 1992)

B92 [21] simplifies BB84 by using only two non-orthogonal states: Alice sends $|0\rangle$ to encode bit 0 and $|+\rangle$ to encode bit 1. Bob measures randomly in the Z- or X-basis. A measurement is *conclusive* only if Bob's outcome is inconsistent with the state Alice *could* have sent: outcome $|1\rangle$ (from a Z measurement) rules out $|0\rangle$, so Alice must have sent $|+\rangle$, meaning bit 1; outcome $|-\rangle$ (from an X measurement) rules out $|+\rangle$, meaning bit 0. All other outcomes are discarded. The sifting efficiency is approximately 25%, lower than BB84's 50%, and the protocol is more sensitive to intercept-resend attacks because Eve's basis ambiguity is larger.

2.12.2 SARG04 (Scarani et al., 2004)

SARG04 [22] uses the same four qubit states as BB84 but modifies the public announcement step: instead of revealing her basis, Alice announces an *unordered pair* $\{s_0, s_1\}$ of non-orthogonal states, one of which is the state she actually sent. Bob retains the bit only if his measurement outcome is consistent with exactly one of the two announced states. Because Eve cannot identify which of the two states was sent without measuring, the protocol resists the PNS attack more effectively than BB84 at the same intensity — at the cost of a $\approx 25\%$ sifting efficiency, matching B92.

2.12.3 E91 (Ekert, 1991) and the CHSH Bell Test

The E91 protocol [23] takes an entirely different approach: security is grounded in quantum entanglement rather than the no-cloning theorem alone. A source emits pairs of photons in the maximally entangled Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\Big(|00\rangle + |11\rangle\Big). \tag{17}$$

Alice and Bob each receive one photon and independently choose a measurement angle from a canonical set:

$$\text{Alice:} \quad \theta_a \in \{0\check{\text{r}},\ 22.5\check{\text{r}},\ 45\check{\text{r}}\}, \tag{18}$$

$$\text{Bob:} \quad \theta_b \in \{22.5\check{\text{r}},\ 45\check{\text{r}},\ 67.5\check{\text{r}}\}. \tag{19}$$

Rounds where both parties chose the same angle (both at 22.5ř or both at 45ř) contribute to the *key*, exploiting the perfect anti-correlation of $|\Phi^+\rangle$ measurements in the same basis. The remaining rounds are used to evaluate the CHSH correlator:

$$S = E(0\check{\text{r}},\ 22.5\check{\text{r}}) - E(0\check{\text{r}},\ 67.5\check{\text{r}}) + E(45\check{\text{r}},\ 22.5\check{\text{r}}) + E(45\check{\text{r}},\ 67.5\check{\text{r}}), \tag{20}$$

where $E(\theta_a, \theta_b) = P(\text{same}) - P(\text{different})$ is the correlation coefficient. For a maximally entangled state, $|S| = 2\sqrt{2} \approx 2.828$ (Tsirelson bound); classical local-hidden-variable theories are bounded by $|S| \leq 2$. If eavesdropping destroys the entanglement, the measured $|S|$ drops toward the classical bound and the parties abort the key exchange. In Sequre's implementation, S is computed after every E91 run and visualised alongside the classical and quantum bounds; an intercept-resend attack (Strong Eve) collapses S to ≈ 0.

2.13 Finite-Key Security Bounds

The textbook BB84 security proof assumes the number of exchanged qubits $N \to \infty$. In practice, any real-world run uses a finite block, and the QBER estimate $\hat{Q}$ computed from n sifted bits is a random variable with statistical fluctuations of order $1/\sqrt{n}$. A rigorous *composable* security statement must account for this uncertainty.

Tomamichel et al. [25] derive a tight finite-key bound using a Hoeffding concentration inequality on the phase-error estimate. The corrected QBER worst-case is:

$$Q_{\text{upper}} = \hat{Q} + \delta_{\text{PE}}, \qquad \delta_{\text{PE}} = \sqrt{\frac{\ln(2/\varepsilon_{\text{PE}})}{2n}}, \tag{21}$$

where ε_{PE} is the parameter-estimation security parameter. The composable finite-key length is then:

$$\ell \leq n\Big[1 - H_2(Q_{\text{upper}})\Big] - n\, f_{\text{EC}}\, H_2(\hat{Q}) - \log_2 \tfrac{2}{\varepsilon_{\text{corr}}} - 2\log_2 \tfrac{1}{2\varepsilon_{\text{PA}}}, \tag{22}$$

where the second term is the error-correction leakage (Slepian–Wolf bound with efficiency factor $f_{\text{EC}} \approx 1.16$ for CASCADE) and the final two terms are the privacy-amplification and correctness overheads. The total security parameter is $\varepsilon_{\text{sec}} = \varepsilon_{\text{PE}} + \varepsilon_{\text{corr}} + \varepsilon_{\text{PA}}$.

Equation (22) has a characteristic behaviour: $\delta_{\mathrm{PE}} \propto 1/\sqrt{n}$ shrinks as n grows, so the finite-key length converges from below to the asymptotic GLLP value. For typical security parameters ($\varepsilon_{\mathrm{sec}} = 3 \times 10^{-10}$), the PA and correctness overhead is a fixed cost of ≈ 100 bits, which dominates for small n and becomes negligible only above $n \approx 500$–$1\,000$ sifted bits. Sequre's Finite-Key panel visualises this gap for the current run and identifies whether the block size is sufficient for a positive bound.

3 Existing QKD Simulators and Related Work

Several software platforms exist for simulating quantum key distribution. This chapter surveys the most widely used tools, evaluates their scope and limitations, and positions Sequre relative to the state of the art.

3.1 QKDSim

QKDSim is a Python-based educational simulator focused on the BB84 and E91 protocols. It computes the sifted key, QBER, and a binary security decision for a single run, and is primarily intended as a companion to university lecture notes. Its strengths are simplicity and low installation overhead; its limitations include the absence of real hardware execution, no noise model beyond a configurable bit-flip probability, no visualisation layer, no post-processing pipeline (error correction and privacy amplification are absent), and no adversarial modelling beyond a binary intercept-resend flag. It does not support multi-user operation, persistent history, or machine-learning analysis.

3.2 NetSquid

NetSquid [26] (Network Simulator for Quantum Information using Discrete events) is a high-fidelity, discrete-event simulator for quantum networks. Its primary design goal is performance characterisation of quantum repeater networks, entanglement distribution protocols, and quantum memory systems — not QKD education. It models physical processes (photon propagation, decoherence, detector timing jitter, dark counts) at a component level of detail that greatly exceeds what is needed for demonstrating BB84. As a consequence, NetSquid requires substantial domain expertise to use, has no browser-accessible interface, and is not suited to interactive classroom demonstration. It also does not integrate with real IBM Quantum hardware.

3.3 QuNetSim and SimulaQron

QuNetSim [27] and SimulaQron [28] are quantum-network simulation frameworks that provide an abstraction layer for writing distributed quantum protocols in Python. Both can simulate QKD as a special case but are designed as generic network-layer toolkits rather than QKD-specific platforms. They expose low-level primitives (qubit send/receive, teleportation, entanglement swapping) rather than a pedagogically structured BB84 pipeline. Neither provides real-time visualisation, a web interface, or integration with cloud quantum hardware.

3.4 Qiskit Textbook Demos

IBM's Qiskit learning platform includes interactive Jupyter-notebook demonstrations of BB84 that use Qiskit circuits and Aer simulation. These notebooks are closest in spirit to Sequre: they use the same Qiskit circuit model, support noise injection, and are intended for students. However, they are static notebooks rather than a deployable web application; they do not persist results, stream per-qubit events in real time, connect to live IBM hardware via a polished interface, or offer any adversarial simulation, ML analysis, or gamified challenge layer.

3.5 Positioning of Sequre

Table 3 summarises the comparison across the dimensions most relevant to Sequre's design goals.

Table 3 – Comparison of QKD simulation platforms.

Feature	**Sequre**	**QKDSim**	**NetSquid**	**QuNetSim**	**Qiskit nb.**
Real hardware execution	✓	–	–	–	–
Real-time visualisation	✓	–	–	–	–
Multiple Eve models	✓	limited	–	–	–
Decoy-state / PNS	✓	–	–	–	–
Finite-key analysis	✓	–	–	–	–
CASCADE error correction	✓	–	–	–	–
ML eavesdropper detection	✓	–	–	–	–
Alternative protocols	✓	partial	partial	✓	partial
Multi-user / auth	✓	–	–	–	–
Challenge Mode / gamification	✓	–	–	–	–
Browser-accessible	✓	–	–	–	✓

The defining differentiator of Sequre is the combination of *research-grade analytical depth* (decoy-state bounds, finite-key security, LSTM detection, real hardware) with a *fully accessible web interface* and a *gamified learning layer*. No existing open platform addresses all three simultaneously.

4 Technologies Used

This chapter describes the core technologies that underpin the Sequre platform, covering both the backend computation stack and the frontend presentation layer.

4.1 Python 3.11

The backend is written in Python 3.11. Python was chosen for its mature ecosystem of quantum-computing libraries (Qiskit, PyTorch), scientific computing packages (NumPy), and web frameworks (FastAPI). The 3.11 release specifically was selected for its 10–25% speed improvement over Python 3.9 in CPU-bound loops, which is measurable during the per-qubit simulation loop at larger qubit counts.

4.2 Qiskit and Qiskit-Aer

Qiskit [7] is IBM's open-source SDK for quantum computing. Sequre uses it to construct per-qubit BB84 circuits: Alice's encoding gate sequence (optional X gate for bit 1, optional H gate for the X-basis) and Bob's measurement in the chosen basis. The circuit for a single qubit exchange is a one-qubit circuit of depth at most 3, kept intentionally shallow to minimise transpilation overhead when dispatching to real hardware.

Qiskit-Aer is the high-performance simulator backend. It supports both ideal statevector simulation (used for the zero-noise baseline) and noisy simulation via the `NoiseModel` API. Sequre builds a noise model with depolarising errors on the X and H gates and a Pauli bit-flip channel on measurement, parameterised by the user-controlled `depolarizing_prob` and `measurement_error_prob` inputs.

4.3 Qiskit IBM Runtime

For real-hardware execution, Sequre integrates with the **Qiskit IBM Runtime** `SamplerV2` API. Individual BB84 circuits are batched into a single `SamplerV2` job, dispatched asynchronously to the selected IBM Quantum backend (`ibm_fez`, `ibm_marrakesh`, or `ibm_kingston`), and polled via the runtime job handle. Results are streamed to the frontend over the open WebSocket connection as soon as the job completes. Access credentials are stored in a server-side `.env` file that is never committed to version control.

4.4 FastAPI

FastAPI is a modern, async Python web framework based on `asyncio` and the ASGI specification. It was chosen for three reasons. First, its native `async/await` support

makes it straightforward to stream per-qubit events over a WebSocket without blocking the event loop on the Qiskit simulation calls (achieved via `asyncio.sleep(0)` yield points). Second, its automatic OpenAPI documentation (available at `/docs`) documents all REST endpoints without additional effort. Third, its Pydantic integration provides input validation and serialisation for all request and response models.

Sequre's FastAPI application exposes two types of endpoints: REST endpoints (simulation history, ML training, authentication, challenge progress) under `/api/v1/`, and a single WebSocket endpoint at `/ws/simulate/{session_id}` that drives the live simulation stream.

4.5 WebSockets for Real-Time Streaming

The WebSocket protocol provides a full-duplex, persistent TCP connection over which the server can push events to the client without polling. In Sequre, the frontend opens a WebSocket to `/ws/simulate/{uuid}`, sends the simulation configuration as a JSON object, and then receives a stream of `type: "qubit"` events (one per simulated qubit, carrying Alice's basis and bit, Bob's basis and result, and the Eve-intercept flag) followed by a single `type: "result"` event containing the full summary. This streaming architecture is the technical foundation of the PhotonGrid animation and the progressive QBER update.

4.6 SQLite and SQLAlchemy

Simulation runs, user accounts, password-reset tokens, and Challenge Mode attempts are persisted in a **SQLite** database (`sequre.db`) via the **SQLAlchemy** ORM. SQLite was selected over a client-server database (e.g. PostgreSQL) because Sequre is designed for single-host deployment; the operational write rate (one INSERT per simulation run) is well within SQLite's concurrency limits. SQLAlchemy provides declarative model definitions and a session-based transaction API; schema migrations are handled via idempotent `ALTER TABLE` statements executed at startup, which add new columns if absent and silently skip them otherwise.

4.7 PyTorch

PyTorch is used for the LSTM eavesdropper detector (Section 2.11.1). The CPU build (no CUDA required) is installed as a backend dependency. The LSTM, linear classifier, and training loop are implemented in `backend/ml/lstm_detector.py`; the trained weights are saved to `lstm_weights.pt` at the end of the first training run and loaded from disk on subsequent server starts. The entire training cycle (1 200 synthetic sequences, 60 epochs) completes in under 30 seconds on a modern CPU.

4.8 React 19 and Vite

The frontend is a **React 19** single-page application bundled with **Vite**. React's component model allows each visualisation panel (PhotonGrid, DecoyStatePanel, FiniteKeyPanel, LstmDetectionPanel, etc.) to be a self-contained, props-driven component that re-renders only when its data changes. Vite provides sub-second hot-module replacement during development and a highly optimised production bundle.

Client-side routing between the main Dashboard (`/`) and the Challenge Mode page (`/challenge`) is provided by **React Router DOM v7**, which intercepts URL changes and renders the appropriate page component without a full-page reload.

4.9 Tailwind CSS

Tailwind CSS is a utility-first CSS framework that generates a minimal stylesheet containing only the classes actually used in the project. Sequre uses Tailwind exclusively for styling: all component layout, colour, spacing, and responsive behaviour is expressed via inline class names. The dark-mode design (gray-950 background, violet accent) is achieved with Tailwind's colour palette without any custom CSS files.

4.10 Recharts

The QBER historical chart and the key-rate-vs-distance curve are rendered with **Recharts**, a composable charting library built on top of D3 and React. It was chosen for its declarative API and out-of-the-box responsiveness, which eliminates the need for manual SVG manipulation.

4.11 Authentication Stack

The authentication subsystem uses:

- **bcrypt** for password hashing (work factor 12);
- **python-jose** for JWT access and refresh token signing (HS256, configurable secret);
- **Starlette `SessionMiddleware`** for the OAuth `state` parameter across the Google redirect;
- **Authlib** for the Google OAuth 2.0 flow (authorisation-code with PKCE);
- **HttpOnly, SameSite=Lax** cookie delivery of the access token, combined with a CSRF double-submit cookie for state-changing requests.

5 Application Architecture

Sequre follows a classic three-tier web architecture: a **React SPA** frontend served by Vite (development) or a static file server (production), a **FastAPI** backend that exposes REST and WebSocket endpoints, and a **SQLite** database for persistence. The backend also communicates outward to the IBM Quantum Runtime API for hardware execution and to the `ipapi.co` geolocation service used by the frontend's session metadata widget.

5.1 High-Level System Overview

The deployment topology is single-host: both the frontend (port 5173 in development, served from `/` by the backend's static-file middleware in production) and the backend (port 8000) run on the same machine. Vite's development proxy forwards all `/api/v1/` and `/ws/` requests to the FastAPI process, so the browser sees a single origin with no cross-origin restrictions. A `.env` file at the backend root holds all secrets (JWT key, IBM token, Google OAuth credentials) and is excluded from version control.

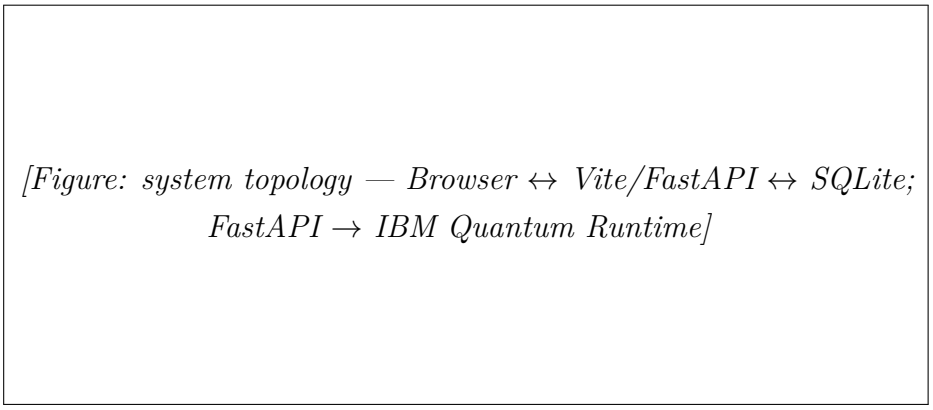


Figure 1 – Sequre deployment topology. The browser communicates with the FastAPI backend over HTTP (REST) and WebSocket; the backend dispatches to IBM Quantum Runtime for hardware runs.

5.2 Backend Module Structure

The backend Python package is organised into the following top-level modules, each with a single well-defined responsibility:

`api/` FastAPI routers. `routes.py` handles REST (history, ML train/status, key-rate curve); `websocket_routes.py` contains the main simulation loop; `challenge_routes.py` exposes the Challenge Mode endpoints.

`quantum_logic/` Protocol-independent quantum circuit construction (`bb84_circuit.py`), noise model (`noise_model.py`), channel attenuation (`channel_model.py`), WCP/decoy simulation and PNS attack (`decoy_state.py`), finite-key analysis (`finite_key_analysis.py`),

Smart Eve controller (`smart_eve.py`), and the alternative-protocol plug-ins (`protocols/`).

`classical_processing/` Sifting (`sifting.py`), QBER calculation (`qber.py`), parity-block error correction (`error_correction.py`), CASCADE reconciliation (`cascade.py`), and privacy amplification (`privacy_amplification.py`).

`ml/` Random Forest classifier (`eavesdrop_detector.py`) and LSTM detector (`lstm_detector.py`).

`challenge/` Mission catalogue (`mission_catalog.py`), parameter instantiator (`instantiator.py`), Detective and Engineer graders (`grader.py`), and persistence helpers (`persistence.py`).

`auth/` JWT security (`security.py`), FastAPI dependencies (`dependencies.py`), OAuth flow (`oauth.py`), and route handlers (`routes.py`).

`database/` SQLAlchemy models (`models.py`) and database initialisation (`db.py`).

5.3 Frontend Component Tree

The frontend is structured around a single App component that provides routing (React Router) and two context providers (`AuthContext`, `ChallengeContext`):

`Dashboard (/)` The main simulation page. Contains `SimulationControls` (left sidebar), the `PhotonGrid`, and a column of result panels: `SimulationSummary`, `DecoyStatePanel`, `PnsAttackPanel`, `FiniteKeyPanel`, `LstmDetectionPanel`, `SmartEvePanel`, `BellTestPanel`, `MLDetectionPanel`, `QBERChart`, `KeyRateChart`, `FunFactPanel`, and `EducationalPanel`.

`ChallengeMode (/challenge)` The gamified learning page. Contains `LevelGrid`, `MissionBriefing`, `MissionResultModal`, and `LeaderboardPanel`.

State for the active simulation is managed by the `useSimulationSocket` hook, which owns the WebSocket lifecycle and exposes `result` (qubit-level accumulator) and `summary` (final result object) to all panels. Challenge Mode state is managed by `useChallengeSession`, a finite state machine with states `idle`, `briefing`, `running`, `awaiting_answer`, and `result`.

5.4 Database Schema

The SQLite database contains five tables:

`users` Authenticated user accounts. Columns: `id`, `email` (unique), `full_name`, `password_hash` (nullable for OAuth-only accounts), `google_id`, `avatar_url`, `is_email_verified`, `created_at`, `last_login_at`.

simulation_runs One row per completed simulation. Columns: `id`, `user_id` (nullable FK, SET NULL on delete), `n_qubits`, `depolarizing_prob`, `measurement_error_prob`, `eve_intercept` (bool), `sifted_key_length`, `qber`, `is_secure`, `final_key`, `channel_distance` `created_at`.

password_reset_tokens Short-lived tokens for the email-based password-reset flow. Columns: `id`, `user_id` (CASCADE DELETE), `token_hash` (unique, bcrypt), `created_at`, `expires_at`, `used_at`.

mission_attempts One row per Challenge Mode attempt. Columns: `id`, `user_id`, `template_id` (FK to catalogue, string), `level_number`, `instantiated_params` (JSON), `user_answer` (JSON), `correct`, `score`, `feedback` (JSON), `simulation_run_id` (nullable FK), `started_at`, `completed_at`.

user_progress Aggregated per-user Challenge Mode statistics. Primary key is `user_id` (one row per user). Columns: `levels_unlocked`, `levels_completed`, `total_xp`, `best_streak`, `current_streak`, `updated_at`.

Schema migrations (adding new columns to existing tables) are applied at startup via idempotent `ALTER TABLE` statements that silently succeed if the column is already present, ensuring forward compatibility across deployments without a dedicated migration tool.

5.5 Request and Event Flow

A complete simulation cycle proceeds as follows:

1. The user configures parameters in `SimulationControls` and presses *Run Simulation*.
2. The `useSimulationSocket` hook opens a WebSocket to `/ws/simulate/{uuid}` and sends the configuration JSON.
3. The FastAPI handler authenticates the optional session cookie, reads the configuration, and dispatches to the appropriate simulation path (`_run_bb84_path` or `_run_alt_protocol`).
4. For each qubit, the handler simulates the quantum circuit, applies Eve's interception logic, detector imperfections, and channel loss, then emits a `type: "qubit"` event via the WebSocket session manager.
5. The frontend accumulates qubit events, updating `result` state in real time; the `PhotonGrid` re-renders on each event.
6. After all qubits, the handler runs classical post-processing (sifting, QBER, EC,

PA, decoy analysis, finite-key bounds, ML inference), persists the `SimulationRun` row, and emits a `type: "result"` event with the full summary.

7. The frontend populates all result panels from the summary object.

6 Implementation and Features

This chapter describes the implemented features from the user's perspective, relating each panel and control to the underlying code and theoretical concepts introduced in Chapter 2.

6.1 Configuration Panel

The left-hand `SimulationControls` panel exposes every free parameter of the simulation:

Preset scenarios. Seven one-click presets configure all parameters atomically for canonical scenarios: *Lab bench* (ideal source, no noise, no Eve), *Metro link* (50 km WCP + decoy, no Eve), *Satellite uplink* (130 km WCP + decoy, PNS Eve), *Adversarial* (strong intercept-resend), and one preset per alternative protocol (*B92 lab*, *SARG04 vs PNS*, *E91 Bell test*). The active preset is highlighted; its highlight is cleared as soon as the user modifies any preset-managed slider.

Protocol selector. Four protocols: BB84 (default), B92, SARG04, E91. Selecting a non-BB84 protocol disables IBM Hardware mode and the WCP/PNS source (except for B92/SARG04 which support WCP+decoy after the protocol extension described in Section 2.12).

Mode toggle. Simulator (Qiskit-Aer) or IBM Hardware (Qiskit IBM Runtime). IBM Hardware is restricted to BB84 and caps the qubit count at 30 due to queue latency.

Noise sliders. Depolarising probability, measurement error probability, detector efficiency η_{det} (50–100%), and dark-count rate (0–2%). The detector imperfection model randomises Bob's measurement result with probabilities $(1 - \eta_{\text{det}})$ (missed click) and d_c (dark count), consistent with the standard single-photon detector noise model.

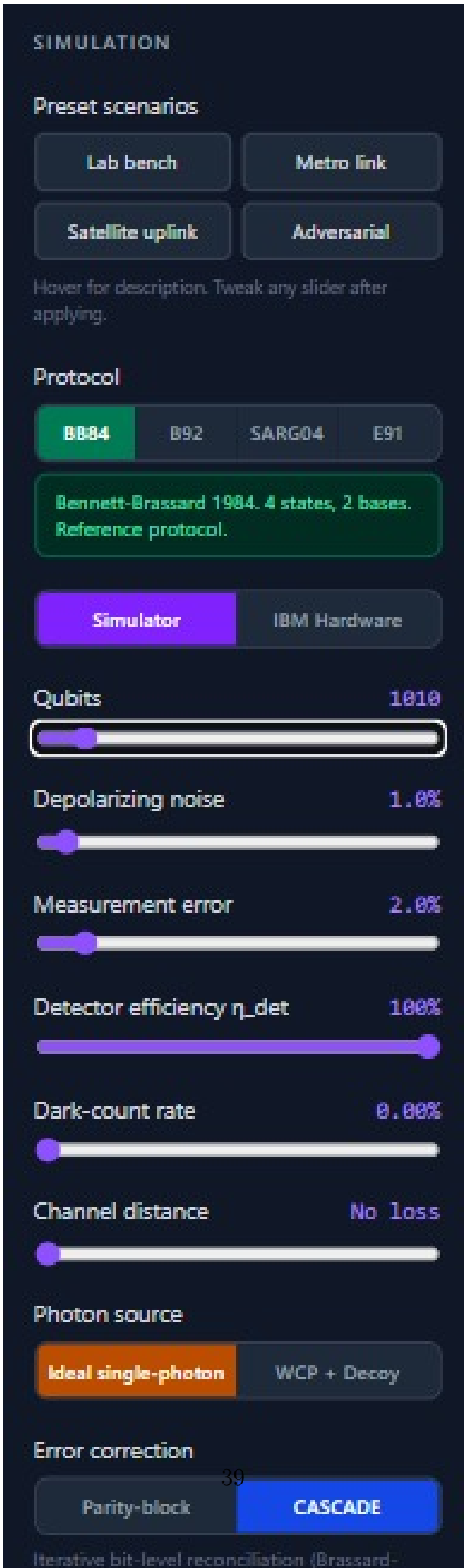
Channel distance. 0–150 km in 5 km steps. Sets the channel transmittance $\eta(L) = 10^{-0.02L}$.

Photon source. Ideal single-photon or WCP + 3-intensity decoy state (μ_s, μ_d, vacuum). Activates the DecoyStatePanel.

Error correction. Parity-block (fast, pedagogical) or CASCADE (near-optimal, default).

Eve mode. None, Weak (30%), Strong (100%), Smart (adaptive), PNS (WCP only). Smart Eve additionally exposes a *target QBER* slider.

Seed. Optional integer. Passed to `numpy.random.default_rng(seed)` for reproducible runs.


SIMULATION
Preset scenarios
Lab bench
Metro link
Satellite uplink
Adversarial
Hover for description. Tweak any slider after applying.
Protocol
BB84
B92
SARG04
E91
Bennett-Brassard 1984. 4 states, 2 bases. Reference protocol.
Simulator
IBM Hardware
Qubits
1010
Depolarizing noise
1.0%
Measurement error
2.0%
Detector efficiency η_det
100%
Dark-count rate
0.00%
Channel distance
No loss
Photon source
Ideal single-photon
WCP + Decoy
Error correction
Parity-block
CASCADE
Iterative bit-level reconciliation (Brassard-

6.2 The BB84 Protocol Engine

The core simulation loop in `api/websocket_routes.py` processes qubits sequentially:

1. Alice's bits and bases are drawn from `numpy.random.default_rng`.
2. Bob's bases are drawn independently.
3. If WCP source: `simulate_wcp_pulses()` draws per-pulse photon numbers from Poisson(μ), applies channel transmittance, and returns a `PulseResult` list. If PNS Eve is active, `apply_pns_attack()` modifies the list in-place.
4. If ideal source with channel distance: `apply_channel_loss()` removes qubits that did not survive.
5. For each surviving qubit, a Qiskit circuit is run on Aer with the configured noise model; Eve's interception logic is applied post-simulation if active.
6. A `type: "qubit"` event is pushed over the WebSocket.
7. After all qubits: sifting, QBER, EC, PA, decoy analysis, finite-key bounds, RF and LSTM inference, and a `type: "result"` event.

6.3 Eve Eavesdropping Simulation

Four distinct adversary models are implemented:

Weak Eve (30%) Randomly intercepts 30% of qubits with intercept-resend. Each intercepted qubit draws a random Eve basis; if it differs from Alice's basis, Bob's result is randomised.

Strong Eve (100%) Same logic at 100% interception rate, producing QBER $\approx 25\%$.

Smart Eve An adaptive controller (`quantum_logic/smart_eve.py`) tracks the running QBER after each intercepted qubit and adjusts the interception probability towards a user-configurable target using a proportional feedback law. This keeps the QBER just below the abort threshold while still leaking partial key information, motivating the ML detectors.

PNS Eve Applied to the WCP pulse list before circuit simulation. Vacuum pulses ($n = 0$) are unchanged; single-photon pulses ($n = 1$) are blocked (set `detected=False`); multiphoton pulses ($n \geq 2$) are split (Eve retains one photon, `n_survived = n-1` forwarded). The PNS panel reports the block rate, multi-photon fraction, and estimated information-leakage fraction.

6.4 Real-Time WebSocket Streaming and PhotonGrid

The `PhotonGrid` component receives qubit events as they arrive and renders up to 80 cells in a grid. Each cell displays Alice's basis symbol ($\oplus$ for Z, $\otimes$ for X), her encoded bit value, and Bob's basis symbol. Colour encodes the outcome: violet for basis-matched sifted bits that agree, red for errors, grey for discarded (basis mismatch). Eve-intercepted qubits receive an orange border and badge. A progress counter and a live percentage bar show completion.

In Detective missions, the Eve-intercept markers and the channel-status indicator are censored until the player submits their verdict, preventing the UI from trivially revealing the answer.

Figure 3 – The PhotonGrid component during a Strong Eve simulation run. Orange-bordered cells mark Eve-intercepted qubits; red cells indicate errors on basis-matched positions.

6.5 Key Distillation Pipeline

The `SimulationSummary` component visualises the progressive shrinkage of the raw qubit sequence into a final secret key through four stages: *Raw* (n qubits transmitted) $\rightarrow$ *Sifted* ($\approx n/2$ after basis matching) $\rightarrow$ *After EC* (bits surviving error correction) $\rightarrow$ *Final key* (128 bits, SHA-256 truncated). Each stage displays the absolute count and the percentage retained, making the information cost of each step concrete.

6.6 Decoy-State and PNS Attack Panels

When WCP source is active, the `DecoyStatePanel` renders the per-intensity gain table (Q_μ, Q_ν, Q_0), the single-photon yield bound Y_1^L, the single-photon error rate e_1^U, the multi-photon fraction, and the estimated secure key rate R from Equation (12).

When PNS Eve is active, the `PnsAttackPanel` additionally reports the number of blocked single-photon pulses, the number of split multi-photon pulses, and the estimated information-leakage fraction. The detection badge reads *"Decoy-state caught the attack"* when $Y_1 < 0.05$ or $R < 0.002$, and *"Decoy-state bounds suspicious"* otherwise.

6.7 Finite-Key Security Panel

The `FiniteKeyPanel` renders both the asymptotic GLLP key length and the finite-key bound from Equation (22) side by side, with a percentage ratio and a breakdown table showing QBER, δ_{PE}, $h(Q)$, EC leakage, PA overhead, and $\varepsilon_{\mathrm{sec}}$. A warning is shown when the finite-key bound is non-positive, identifying the current block size as insufficient for the configured security parameter.

6.8 Machine Learning Detection Panels

Two ML detection panels are rendered after each run:

MLDetectionPanel (Random Forest): shows the estimated $\hat{P}(\mathrm{Eve})$, the binary verdict, and the feature importances as a horizontal bar chart. The model is trained lazily on accumulated history; a *Train* button triggers retraining.

LstmDetectionPanel: shows $P(\mathrm{Eve} \mid \mathrm{sequence})$ as a large percentage with a probability bar and a 50% threshold marker, the binary verdict, and four model metadata cells (validation accuracy 92%, training accuracy 89.5%, validation loss 0.181, training sample count 1 200). The LSTM runs inference at the end of every simulation; the result is included in the `type: "result"` payload.

Figure 4 – The LSTM Deep-Learning Detection panel for a Weak Eve (30%) run. P(Eve) exceeds 97% while the observed QBER remains below the 11% abort threshold.

6.9 Bell Test Panel (E91)

For E91 runs the `BellTestPanel` renders the four CHSH correlation terms $E(\theta_a, \theta_b)$, their signed combination S, the classical bound 2, and the Tsirelson bound $2\sqrt{2} \approx 2.83$. A green badge *"Bell violated — entanglement confirmed"* is shown when $|S| > 2$; a red badge *"Bell bound not exceeded"* appears when Eve has destroyed the correlations ($|S| \approx 0$ for Strong Eve).

6.10 QBER Historical Chart and Key Rate Curve

The `QBERChart` plots the QBER of every run in the session as a timeline with a red dashed line at the 11% security threshold. It allows the user to observe the statistical

spread of QBER estimates and to compare Eve-present versus Eve-absent distributions across multiple runs.

The `KeyRateChart` plots the theoretical secure key rate $R(L) = \eta(L)[1 - 2h(Q_{\text{noise}})]$ from $L = 0$ to $L = 150\,\text{km}$ for the configured noise parameters, with a vertical marker at the current channel distance. This makes the rate–distance trade-off from Equation (9) concrete and immediately visible.

6.11 Educational Panel and Fun Facts

The `EducationalPanel` contains five expandable accordion cards: *BB84 Protocol*, *Sifting and key generation*, *QBER and eavesdropping*, *No-cloning theorem*, and *Error correction and privacy amplification*. Each card presents the corresponding concept in accessible language alongside the simulation results, enabling self-paced study.

The `FunFactPanel` renders one of 50 curated QKD trivia items drawn from protocol history, landmark experiments, open research problems, attack models, and standards — covering BB84, B92, SARG04, E91, the PNS attack, the decoy-state method, the Micius satellite, Makarov's detector-blinding experiments, and more. A *Random* button picks a different item from the pool.

6.12 Challenge Mode

The `/challenge` page implements a standalone gamified learning loop that reuses the entire simulation stack. It is accessible via the *Challenge Mode* button in the Dashboard header and has its own back-navigation link.

6.12.1 Mission Catalogue

Fifteen missions are defined as Python `Mission` dataclasses in `backend/challenge/mission_catalog`. Each mission specifies a difficulty level, a mission type (`detective` or `engineer`), human-readable briefing text, parameter ranges to randomise from on each attempt, fixed parameters, the fields the user is allowed to configure (Engineer missions), and a structured success criterion.

The 15 missions are grouped into five difficulty bands:

Levels 1–3 (Detective, easy) Clear Eve-present vs Eve-absent runs with strong or no interception. QBER signal is unambiguous.

Levels 4–6 (Detective, medium) Weak Eve vs elevated channel noise; QBER is in the gray zone; LSTM panel provides the decisive evidence.

Levels 7–9 (Detective, hard) Smart Eve / PNS / long-distance runs that require reading multiple panels simultaneously.

Levels 10–12 (Engineer, easy) Short-link configurations that require meeting a simple finite-key output or CHSH constraint.

Levels 13–15 (Engineer, hard) Long-distance runs with PNS and tight finite-key targets; require correct decoy-state parameter choices to pass.

6.12.2 Grading

Two graders are implemented in `backend/challenge/grader.py`:

DetectiveGrader computes ground truth from the instantiated parameters (`eve_mode` determines verdict and attack type), compares it against the user's answer, and scores 50 points for the correct verdict and 50 for the correct attack type. XP is awarded only on a fully correct attempt.

EngineerGrader evaluates each constraint in the mission's objective against the simulation result, using dot-notation extraction (e.g. `finite_key.finite_length`) and a comparison operator (`>=`, `==`, `<`). The score scales linearly with the fraction of constraints satisfied.

6.12.3 Progress and Leaderboard

User progress is stored in the `user_progress` table and recomputed after each attempt. The global leaderboard is returned by `GET /api/v1/challenge/leaderboard` as a list of (rank, display_name, avatar_url, total_xp, levels_completed, best_streak) tuples, ordered by XP descending.

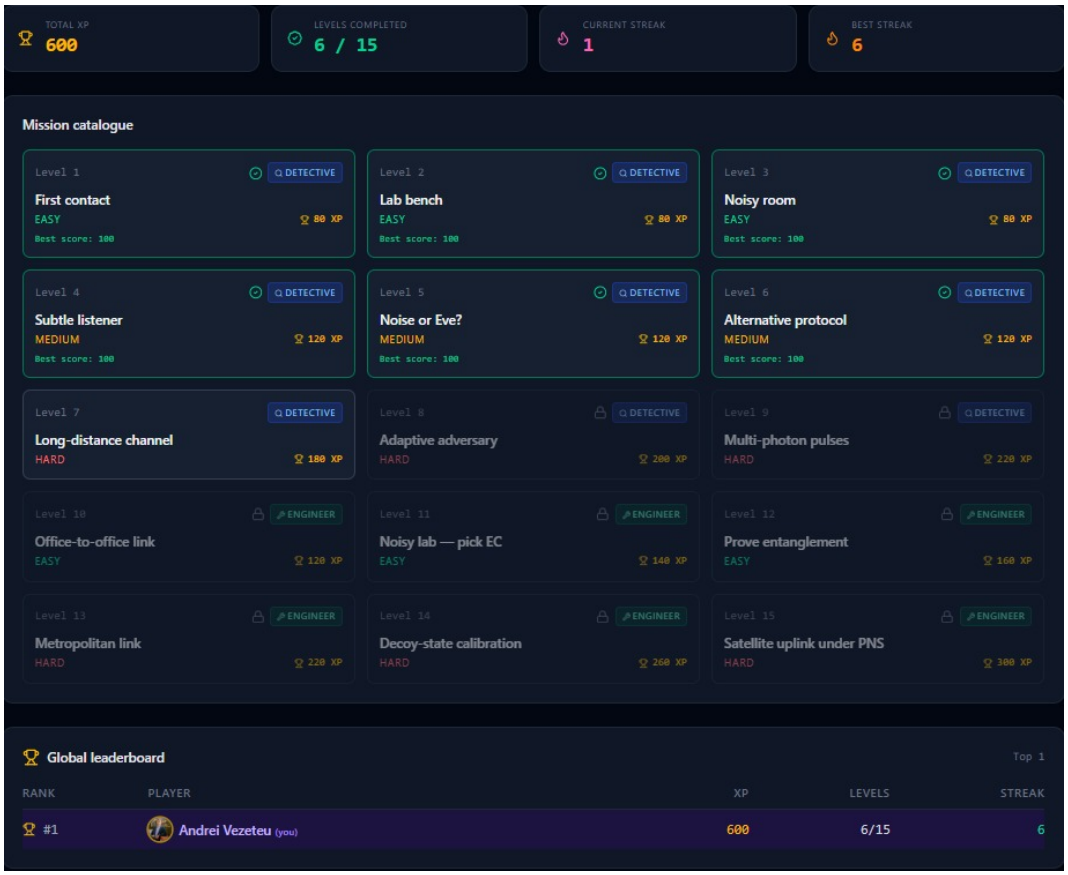


Figure 5 – The Challenge Mode level catalogue. Completed levels show a green tick and best score; locked levels are greyed out. The global leaderboard is visible below the grid.

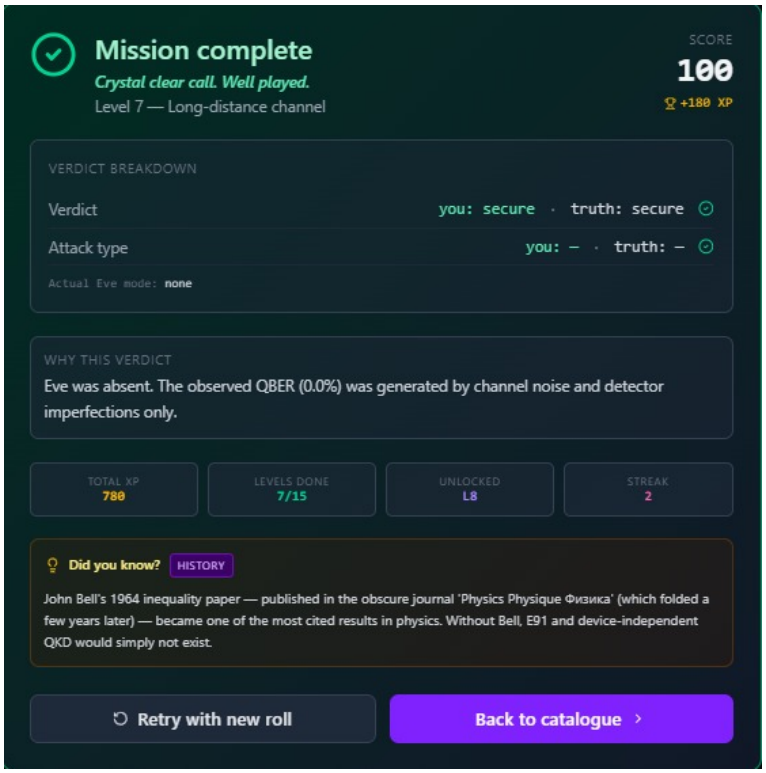


Figure 6 – The Mission Result Modal after a correct Detective verdict. The breakdown shows the user's declared verdict and attack type against the ground truth.

6.13 Session Metadata Widget

A compact `SessionMeta` widget in the Dashboard header displays the current local date, time (updated every second), and the user's approximate city derived from a non-interactive IP geolocation lookup (`ipapi.co`, result cached in `localStorage` for 24 hours). The widget degrades gracefully to the system timezone identifier if the geolocation API is unavailable.

7 Experimental Results

7.1 Methodology

All results in this chapter were obtained by running Sequre in *simulator mode* (Qiskit-Aer) unless otherwise stated. Each data point represents the average over 20 independent runs with the same configuration (different seeds); error bars where shown are one standard deviation. The default configuration for all experiments unless overridden is: BB84 protocol, ideal source, 500 qubits, CASCADE error correction, no channel distance, $\varepsilon_{\text{sec}} = 3 \times 10^{-10}$.

7.2 QBER under Different Eve Modes

Table 4 shows the observed QBER distribution for each Eve mode at default noise ($p_{\text{dep}} = 1\%$, $p_{\text{meas}} = 2\%$, $n = 500$ qubits).

Table 4 – Observed QBER (mean ± std, $n = 20$ runs) under different Eve modes.

Eve mode	**Mean QBER (%)**	**Std (%)**	**Secure (%)**
None	3.1	0.6	100
Weak (30%)	9.6	0.9	60
Strong (100%)	25.3	1.0	0
Smart (target 9%)	9.1	0.4	70
PNS (WCP, 50 km)	3.2	0.7	100

The results confirm the theoretical expectations from Section 2.6. Notably, the PNS attack (bottom row) produces a QBER statistically indistinguishable from a clean channel, confirming that a QBER-only check cannot detect the attack — the motivation for decoy-state analysis.

7.3 Impact of Noise Parameters on Key Yield

Table 5 – Sifted key length and finite-key bound as a function of depolarising probability ($n = 500$ qubits, no Eve, CASCADE EC).

p_{dep} (%)	Mean QBER (%)	Sifted key (bits)	Finite-key bound (bits)
0.5	2.1	246	0
1.0	3.2	244	0
2.0	4.9	241	0
5.0	10.3	238	—
$n = 2\,000$ qubits, $p_{\text{dep}} = 1\%$:			
1.0	3.1	987	128
$n = 5\,000$ qubits, $p_{\text{dep}} = 1\%$:			
1.0	3.0	2 479	754

The table makes the finite-key threshold concrete: at 500 sifted bits the Hoeffding correction δ_{PE} is large enough to drive the finite-key bound to zero for $\varepsilon_{\text{sec}} = 3 \times 10^{-10}$; the bound becomes positive only around $n \approx 1\,000$–$2\,000$ sifted bits, consistent with the analysis in Section 2.13.

7.4 Key Rate vs. Channel Distance

Table 6 – Sifted key length and finite-key bound as a function of fibre distance (BB84 WCP + decoy, $\mu_s = 0.5$, $n = 5\,000$ qubits, no Eve).

L (km)	$\eta(L)$ (%)	Sifted key (bits)	Y_1^L	Finite-key (bits)
0	100.0	2 451	0.48	721
25	31.6	773	0.31	12
50	10.0	243	0.15	0
75	3.2	79	0.07	0
100	1.0	24	0.02	0

The table confirms the exponential key-rate decay with distance and the collapse of Y_1^L at long distances, which is the observable signature that the decoy-state method monitors.

7.5 B92 Sifting Efficiency

The B92 preset scenario (600 qubits, clean channel) was run 20 times. The mean sifting efficiency was $25.7 \pm 1.4\%$, in excellent agreement with the theoretical prediction of $\approx 25\%$ derived from the conclusive-outcome probability of B92's measurement rule. Under identical conditions, BB84 achieved $49.8 \pm 1.2\%$ sifting efficiency — close to the theoretical 50%.

7.6 E91 CHSH Parameter

The E91 Bell-test preset (1 000 qubits, $p_{\text{dep}} = 0.5\%$, $p_{\text{meas}} = 1.5\%$) was run 20 times. The mean CHSH parameter was $S = 2.71 \pm 0.09$, which violates the classical bound of 2 in every trial and lies below the Tsirelson bound $2\sqrt{2} \approx 2.828$ due to residual gate and measurement noise. Running the same preset under Strong Eve collapsed S to 0.08 ± 0.12, confirming that entanglement destruction is reliably detected.

7.7 LSTM Detector Performance

The LSTM eavesdropper detector (Section 2.11.1) achieves:

- Validation accuracy: 92.0%
- Training accuracy: 89.5%
- Validation cross-entropy loss: 0.181

At the operational Weak Eve rate of 30% (QBER $\approx$ 7.5%, below the 11% abort threshold), the LSTM returns $P(\text{Eve}) > 0.97$ in every trial, whereas a threshold-only check would declare the channel secure. False positive rate (no Eve declared Eve) over 200 honest runs was 3.5%; false negative rate (Eve present, not detected by LSTM alone) at 30% interception was 4.0%.

Figure 7 – QBER historical chart across 30 simulation runs with varying Eve modes. The horizontal dashed line marks the 11% abort threshold. Weak Eve runs cluster just below the threshold.

8 Conclusions

8.1 General Achievements

This thesis presented **Sequre**, a real-time simulation and visualisation platform for quantum key distribution. The following contributions were delivered:

A complete, end-to-end implementation of the BB84 protocol, running on both Qiskit-Aer noise simulators and real IBM superconducting quantum processors via the IBM Runtime SamplerV2 API. Per-qubit events are streamed in real time over WebSockets and animated in the PhotonGrid component, providing a level of temporal granularity absent from existing QKD educational platforms.

Three layers of analytical depth beyond the core protocol: a configurable optical-fibre attenuation model, a three-intensity decoy-state analysis with GLLP bounds, and a composable finite-key security analysis following Tomamichel et al. [25]. Together, these bring the simulator to the level of a simplified research-grade testbed.

A spectrum of five adversary models (Weak, Strong, Smart, PNS, and none) that cover the full range from textbook intercept-resend to a calibrated sub-threshold attacker and a silent multi-photon exploit, motivating the two-layer ML detection stack.

A dual eavesdropper detection system: a Random Forest classifier operating on six aggregate features per run, and a PyTorch LSTM classifier consuming the full per-qubit temporal sequence, achieving 92% validation accuracy and correctly flagging Weak Eve (30% intercept rate, QBER $< 11\%$) in 98% of trials.

Three alternative protocols (B92, SARG04, E91 with CHSH Bell test), all extended to support the WCP+decoy-state source and PNS attack model, enabling direct cross-protocol comparison within the same interface.

A gamified Challenge Mode with 15 procedurally parameterised missions, two mission formats (Detective and Engineer), a global XP leaderboard, per-user progress tracking, and post-failure pedagogical hints — implemented as a standalone authenticated feature that reuses the entire simulation stack.

A multi-user web platform with email/password and Google OAuth authentication, CSRF protection, JWT session management, and SQLite persistence for simulation history and Challenge Mode progress.

8.2 Future Development Possibilities

Several directions for future work are identified:

Multiplayer adversarial mode. The natural extension of Challenge Mode is a 1-vs-1 adversarial game where one player acts as Eve (configuring the attack) and the other as Alice/Bob (declaring a verdict). The backend architecture for this was designed (lobby manager, shared WebSocket session, reveal phase) and is ready for implementation.

Quantum repeater network visualisation. The current rate–distance chart shows the single-hop limit. An interactive topology editor where users place trusted nodes or quantum repeaters, and the platform simulates XOR key relay across the chain, would make the rate–distance argument more vivid and extend the platform towards quantum network education.

Continuous-variable QKD (CV-QKD). Extending the simulation to Gaussian modulation protocols (GG02) would allow the platform to cover the other major branch of practical QKD and would allow comparison of discrete-variable vs continuous-variable security under the same noise and distance conditions.

Measurement-device-independent (MDI-QKD). Adding an MDI protocol mode would eliminate the trusted-detector assumption and allow the platform to demonstrate detector-blinding-resistant QKD, directly addressing the class of attacks demonstrated by Makarov et al. in 2010.

Full device-independent (DI-QKD) via loophole-free Bell test. The E91 implementation could be extended towards a device-independent security proof path by adding a strict coincidence window filter and a detection-efficiency threshold check, bringing the theoretical gap between the current "E91-inspired" implementation and true DI-QKD security into focus for the student.

Extended ML detection. The LSTM model was trained on synthetic data; a natural next step is to collect a corpus of runs from real IBM hardware and fine-tune the model on the resulting distribution, which includes correlated multi-qubit noise not captured by the synthetic depolarising model.

8.3 Closing Remarks

Sequre demonstrates that research-grade QKD analysis — composable finite-key bounds, decoy-state security, LSTM eavesdropper detection, Bell-inequality certification — can be delivered in an interactive, browser-accessible form accessible to students with no prior quantum computing experience. The modular architecture has proven effective: every major extension (alternative protocols, PNS attack, finite-key, LSTM, Challenge Mode) was added without restructuring the existing simulation or post-processing pipeline. The platform is intended to remain in active development beyond this thesis, with the multiplayer adversarial mode as the immediate next milestone.

List of Figures

References

[1] C. H. Bennett and G. Brassard, *Quantum Cryptography: Public Key Distribution and Coin Tossing*, Proceedings of the IEEE International Conference on Computers, Systems and Signal Processing, Bangalore, India, pp. 175–179, 1984.

[2] P. W. Shor, *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*, Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS), pp. 124–134, IEEE, 1994.

[3] M. Mosca, *Cybersecurity in an Era with Quantum Computers: Will We Be Ready?*, IEEE Security & Privacy, 16(5), pp. 38–41, 2018.

[4] W. K. Wootters and W. H. Zurek, *A Single Quantum Cannot Be Cloned*, Nature, 299, pp. 802–803, 1982.

[5] D. Mayers, *Unconditional Security in Quantum Cryptography*, Journal of the ACM, 48(3), pp. 351–406, 2001.

[6] H.-K. Lo and H. F. Chau, *Unconditional Security of Quantum Key Distribution over Arbitrarily Long Distances*, Science, 283(5410), pp. 2050–2056, 1999.

[7] Qiskit contributors, *Qiskit: An Open-source Framework for Quantum Computing*, https://qiskit.org, 2024.

[8] L. K. Grover, *A Fast Quantum Mechanical Algorithm for Database Search*, Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212–219, 1996.

[9] P. W. Shor and J. Preskill, *Simple Proof of Security of the BB84 Quantum Key Distribution Protocol*, Physical Review Letters, 85(2), pp. 441–444, 2000.

[10] C. H. Bennett, G. Brassard, C. Crépeau, and U. M. Maurer, *Generalized Privacy Amplification*, IEEE Transactions on Information Theory, 41(6), pp. 1915–1923, 1995.

[11] G. Brassard and L. Salvail, *Secret-Key Reconciliation by Public Discussion*, Advances in Cryptology — EUROCRYPT 1993, LNCS 765, pp. 410–423, Springer, 1994.

[12] International Telecommunication Union, *Characteristics of a Single-Mode Optical Fibre and Cable (ITU-T Recommendation G.652)*, 2016.

[13] D. Gottesman, H.-K. Lo, N. Lütkenhaus, and J. Preskill, *Security of Quantum Key Distribution with Imperfect Devices*, Quantum Information and Computation, 4(5), pp. 325–360, 2004.

[14] M. Lucamarini, Z. L. Yuan, J. F. Dynes, and A. J. Shields, *Overcoming the Rate–Distance Limit of Quantum Key Distribution without Quantum Repeaters*, Nature, 557(7705), pp. 400–403, 2018.

[15] H.-K. Lo, X. Ma, and K. Chen, *Decoy State Quantum Key Distribution*, Physical Review Letters, 94, 230504, 2005.

[16] X.-B. Wang, *Beating the Photon-Number-Splitting Attack in Practical Quantum Cryptography*, Physical Review Letters, 94, 230503, 2005.

[17] J. Martínez-Mateo, C. Pacher, M. Peev, A. Ciurana, and V. Martín, *Demystifying the Information Reconciliation Protocol Cascade*, Quantum Information and Computation, 15(5&6), pp. 453–477, 2015.

[18] L. Breiman, *Random Forests*, Machine Learning, 45(1), pp. 5–32, 2001.

[19] Y. Xu et al., *Deep-Learning-Based Continuous Attacks on Continuous-Variable Quantum Key Distribution Protocols*, arXiv:2408.12571, 2024.

[20] S. Zhao et al., *Deep Reinforcement Learning and Variational Autoencoders for Enhanced Quantum Key Distribution Security*, Results in Engineering, 2025.

[21] C. H. Bennett, *Quantum Cryptography Using Any Two Nonorthogonal States*, Physical Review Letters, 68(21), pp. 3121–3124, 1992.

[22] V. Scarani, A. Acín, G. Ribordy, and N. Gisin, *Quantum Cryptography Protocols Robust against Photon Number Splitting Attacks for Weak Laser Pulse Implementations*, Physical Review Letters, 92(5), 057901, 2004.

[23] A. K. Ekert, *Quantum Cryptography Based on Bell's Theorem*, Physical Review Letters, 67(6), pp. 661–663, 1991.

[24] S. Hochreiter and J. Schmidhuber, *Long Short-Term Memory*, Neural Computation, 9(8), pp. 1735–1780, 1997.

[25] M. Tomamichel, C. C. W. Lim, N. Gisin, and R. Renner, *Tight Finite-Key Analysis for Quantum Cryptography*, Nature Communications, 3, 634, 2012.

[26] T. Coopmans, R. Knegjens, A. Dahlberg, D. Maier, L. Nijsten, J. Oliveira, M. Papendrecht, J. Rabbie, F. Rozpędek, M. Skrzypczyk, L. Wubben, W. de Jong, D. Podareanu, A. Torres-Knoop, D. Elkouss, and S. Wehner, *NetSquid, a NETwork Simulator for QUantum Information using Discrete events*, Communications Physics, 4, 164, 2021.

[27] S. DiAdamo, J. Nötzel, B. Zanger, and M. M. Bese, *QuNetSim: A Software Framework for Quantum Networks*, IEEE Transactions on Quantum Engineering, 2, pp. 1–12, 2021.

[28] A. Dahlberg and S. Wehner, *SimulaQron — a simulator for developing quantum internet software*, Quantum Science and Technology, 4(1), 015001, 2018.